One Job Sequence for an Eight-Stage Line

OM 522 team project, option 6: prototype check-in

Muting (Don) Ma

November 3, 2026

The rule to beat reproducing the worked example, an independent checker, the researched method, and its first results.

Repository: github.com/mutingma123/OM522Project

Ten minutes in five parts

1. The decision (0:50)
2. Rule to beat and checker (2:40)
3. Researched method (2:10)
4. First implementation against the rule (1:40)
5. Walk-throughs, one per member (1:50)
6. Sources

Times are the planned share of the 10-minute limit; the title and closing take 30 seconds.

A planner picks one job order for all eight stages

The decision

- Jobs are released over time, each with a due date and a weight.
- Every job visits stages S1 to S8 in order, and jobs cannot pass one another, so one sequence holds at every stage.
- Primary measure: total weighted tardiness. Secondary measure: total waiting between stages.

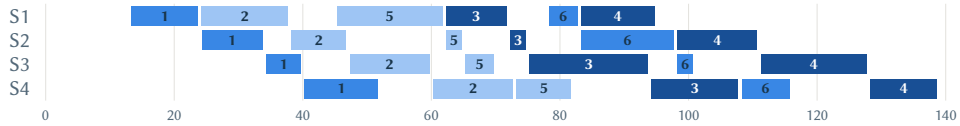
Assumptions we added

- Every stage is free at time zero.
- The sequence is fixed before the first job starts and is not revised.
- Among sequences with equal weighted tardiness, less waiting is better.

A job that is slow at a later stage holds up every job behind it, so a good order depends on more than due dates.

Rule to beat: earliest due date gives 281 on the hand-checkable case

Sort jobs by due date (ties by job identifier) and use that order at every stage; each operation starts at the later of the job's previous completion (or its release) and the stage's previous completion.



Bars carry the job number; shading is the job's weight: 1 2 4. Minutes on the axis.

J001 starts S1 at its release 13; J002, released at 8, waits for S1 until 24; S1 then sits idle until J005 is released at 45.

Weighted tardiness 281 and waiting 13: the decision is identical to the worked example.

Terminal:

```
pixi run test
pixi run solve data/hand_checkable out/hand_rule --method rule
pixi run check data/hand_checkable out/hand_rule/decision.json --expect data/worked-example/metrics.json
```

Separate code confirms each decision from the input files

What the checker confirms

- Each job once in the sequence; each job-stage pair once in the schedule.
- Each duration equals the processing time within 10^{-8}.
- Release at S1, then stage order, for every job.
- One job at a time per stage, in the sequence's order.

What it reported

- Worked example: feasible; every recomputed measure matches `metrics.json`.
- Fresh copy of `development_medium` under EDD: feasible, weighted tardiness 2,371.
- Edge case: rejected with `line 2: 0 is not positive`, no `decision.json`, exit status 1.

It recomputes every measure and imports nothing from the solver, so a method's timing error cannot repeat here.

Terminal:

```
cp -r data/development_medium ~/fresh_medium
pixi run solve ~/fresh_medium out/medium_rule --method rule
pixi run check ~/fresh_medium out/medium_rule/decision.json
pixi run solve data/edge_case out/edge
```

Researched method: iterated greedy for weighted tardiness

Source

Ruiz and Stützle's IG_RS_LS for flow shops with total weighted tardiness [1], which adapts their iterated greedy for the makespan [2].

Why it should beat EDD here

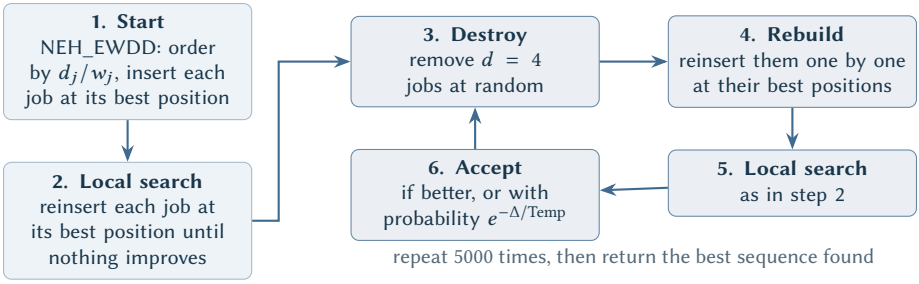
- EDD sorts on due dates alone and ignores weights, release times, and processing times.
- Iterated greedy compares candidate sequences by their actual weighted tardiness over all eight stages, so a heavy job can move ahead of a light one.

Evidence in the sources

- Its weighted due date start, NEH_EWDD, beat plain NEH on 470 of 480 instances [1].
- IG_RS_LS had the lowest average deviation from the best known solutions in all four weighted tardiness instance sets [1].
- In a review of 40 tardiness methods, insertion-based heuristics did best and EDD ranked below its NEH variant [3].

Neither source has release times; our timing includes them in every evaluation.

How it builds, improves, and stops



- Solution: a permutation of the jobs.
- Compare: weighted tardiness, then waiting.
- $d = 4$, $T = 0.5$, Temp $= T \sum_{j,k} p_{jk}/(10\,n\,m)$ [1].
- Stop: 5000 iterations, seed 522; a 300 s cap is only a safety limit.
- Start from the better of NEH_EWDD and EDD, so the result is never worse than the rule.

On the hand-checkable case it reaches the optimum: 233 against 281



Moving the weight-1 job J005 last lets the weight-4 jobs finish earlier: J003 at 97 instead of 108 and J004 at 119 instead of 139.

Enumerating all 720 sequences confirms 233 is optimal; waiting rises from 13 to 37 minutes.

Lower weighted tardiness than the rule on all four instances

		Weighted tardiness		Waiting (min)		
Instance	Jobs	EDD	Iterated greedy	EDD	Iterated greedy	Seconds
hand_checkable	6	281	233	13	37	0.2
development_small	15	1,417	21	732	252	0.5
development_medium	45	2,371	58	2,040	794	3.2
development_large	120	23,015	4,383	7,863	3,005	37.2

- All eight decisions pass the checker. Seconds are for iterated greedy; the rule takes under a millisecond. Intel Core Ultra 7 155U laptop.
- At 120 jobs, 5000 iterations have not converged: five other seeds reached 4,159 to 4,306.
- For the final submission: how results change with size, and a stress test of both fixed plans on 30 saved processing-time scenarios.

Terminal:

```
pixi run solve ~/fresh_medium out/medium_ig
pixi run check ~/fresh_medium out/medium_ig/decision.json
```

Walk-through: the checker's violation messages

`src/pfsp_check/check.py`, drafted by Claude Code

What it does

Reads only the instance CSVs and `decision.json`, names every job and stage that breaks a constraint, and recomputes the measures.

What was wrong in the agent's version

Messages printed every time as a float, such as `starts at 23.0 before it completes S1 at 24.0`. The data are whole minutes, so the messages did not read like the input and did not match the test we wrote.

How we checked it

`tests/test_checker.py` damages the worked example in six ways and asserts each message; the fix is commit `9db7864`.

Walk-through: the shortcut in the insertion search

`best_insertion` in `src/pfsp/timing.py`, drafted by Claude Code

What it does

Tries one job at every position of a partial sequence and returns the best, timing the jobs ahead of each position only once.

What was unclear in the agent's version

It abandons a position as soon as the running totals exceed the best position found. Could that skip the true best position?

How we checked it

The shortcut is safe because weighted tardiness and waiting only grow as jobs are appended. `test_best_insertion_matches_brute_force` compares it with evaluating every position on 20 random cases.

Walk-through: the method's start and temperature

`src/pfsp/ig.py`, drafted by Claude Code

What it does

Builds the NEH_EWDD start, then runs the destroy, rebuild, local search, and accept loop.

What was wrong in the agent's first plan

It ordered the start by plain due date and used $T = 0.4$ from the makespan paper [2]. The weighted tardiness paper orders by d_j / w_j and uses $T = 0.5$ [1].

How we checked it

Reading pages 1146 to 1148 of the 2008 paper; on the hand-checkable case the method matches the optimum from enumerating all 720 sequences, and it beats the rule on all four instances in `tests/test_ig.py`.

What this check-in shows

- The rule to beat reproduces the worked example exactly, and the independent checker confirms it on the worked example and a fresh development instance.
- Iterated greedy [1] beats the rule on weighted tardiness on all four instances, and the checker confirms every decision.
- It costs seconds of computation and, on the six-job case, more waiting.
- Next: compare the methods by instance size and stress-test both fixed plans under processing-time variability.

Sources

[1] Rubén Ruiz and Thomas Stützle.

An Iterated Greedy heuristic for the sequence dependent setup times flowshop problem with makespan and weighted tardiness objectives.

European Journal of Operational Research, 187(3):1143-1159, 2008.

doi:10.1016/j.ejor.2006.07.029.

[2] Rubén Ruiz and Thomas Stützle.

A simple and effective iterated greedy algorithm for the permutation flowshop scheduling problem.

European Journal of Operational Research, 177(3):2033-2049, 2007.

doi:10.1016/j.ejor.2005.12.009.

[3] Eva Vallada, Rubén Ruiz, and Gerardo Minella.

Minimising total tardiness in the m-machine flowshop problem: A review and evaluation of heuristics and metaheuristics.

Computers & Operations Research, 35(4):1350-1373, 2008.

doi:10.1016/j.cor.2006.08.016.